

Auditoría Técnica Holística e Integral — ObSave v1.1.3

Ad Astra Forge · Plugin ObSave **Fecha:** 10 de septiembre de 2026 **Commit auditado:** `afaa5a1` (release v1.1.3) **Alcance:** Ciclo de vida de creación de carpetas, estados del Ledger en modo offline / auto-sync desactivado, integridad del manifiesto y barrido de puntos ciegos **Nota:** Informe de solo diagnóstico. **No se modificó código ni se publicó versión alguna.**

Resumen ejecutivo

Se identificaron **21 hallazgos**, de los cuales **3 son críticos**. El más grave (**H-01**) no fue el reportado por el usuario: la sincronización **multi-dispositivo está funcionalmente rota**, porque un dispositivo secundario marca como sincronizado todo lo que el manifiesto remoto declara en `S` sin descargar ni un solo archivo.

ID	Área	Severidad	Síntoma / causa raíz
H-01	<code>SyncEngine.applyRemoteManifestChanges</code>	Crítica	Entradas remotas en <code>S</code> se copian al ledger local sin verificar existencia en disco → segundo dispositivo nunca descarga nada
H-02	Botón «Generar carpetas» sin auto-sync	Crítica	<code>registerRemoteFolder</code> marca <code>S</code> sin propagar el manifiesto remoto; con auto-sync OFF el ciclo nunca corre → <code>S</code> local huérfano y motor que ignora las carpetas
H-03	<code>pushRemoteFolder</code> (GitHub)	Crítica	<code>return entry.remoteId ?? path</code> → <code>S</code> con <code>remoteId</code> = ruta literal y cero peticiones HTTP
H-04	<code>LedgerManager.trackFolder</code>	Alta	Retorno temprano si existe entrada: carpeta en <code>D</code> recreada localmente sigue en <code>D</code> → borrado remoto de carpeta viva
H-05	<code>folderPathCache</code> (Drive)	Alta	<code>resolveOrCreateFolderPath</code> puede devolver id cacheado sin HTTP; id obsoleto si la carpeta se borró en Drive
H-06	<code>FileStatusDecorator.resolveFolderStatus</code>	Alta	Carpeta sin entrada en ledger y sin hijos pendientes se pinta ● verde
H-07	<code>main.disconnectProvider</code>	Alta	No limpia <code>ledger.json</code> (solo <code>settings.syncedLedger</code>) → tras reconectar a otra cuenta quedan <code>S</code> con <code>remoteId</code> ajenos
H-21	<code>LedgerManager.load</code>	Alta	JSON corrupto degrada en silencio a manifiesto vacío, sin backup ni aviso

H-08	deleteRemotePath	Media-Alta	Sin remoteId no borra nada pero el llamador elimina la entrada → huérfano remoto silencioso
H-09 ... H-20	Varios	Media / Baja	Ver §4

1. Estado inicial incorrecto de carpetas creadas sin auto-sync

1.1 Flujo exacto del botón «Generar carpetas de la bóveda»

```
sequenceDiagram
    participant UI as ObSaveSettingTab
    participant VS as vaultStructure.ts
    participant OB as Obsidian Vault
    participant MAIN as main.ts
    participant SE as SyncEngine
    participant VFS as vaultFolderSync.ts
    participant GD as GoogleDriveProvider
    participant LM as LedgerManager

    UI->>VS: generateVaultTemplateFolders(app)
    VS->>OB: createFolder("05_Archivadas")
    OB-->>MAIN: vault.on("create")
    MAIN->>LM: trackFolder(path) → status C
    UI->>UI: if (!isProviderConfigured) return
    Note over UI: NO evalúa autoSyncEnabled
    UI->>SE: syncTemplateFoldersToCloud()
    SE->>VFS: syncTemplateFoldersToGoogleDrive(provider, cb)
    VFS->>GD: resolveOrCreateFolderPath(root, "05_Archivadas")
    GD-->>VFS: folderId (HTTP o CACHE)
    VFS->>SE: cb("05_Archivadas", folderId)
    SE->>LM: markSynchronized(...) → status S
    SE->>LM: save() (solo disco local)
    Note over SE: El manifiesto remoto .obsave/ledger.json NO se sube
```

1.2 Por qué termina en S y no en C

Causa raíz directa: `ObSaveSettingTab.generateAndSyncVaultFolders()` invoca `syncTemplateFoldersToCloud()` con una única guarda, `isProviderConfigured(settings)`. **No consulta `autoSyncEnabled` ni el estado del temporizador.** Desde la v1.1.3, esa ruta ejecuta el callback `registerRemoteFolder`, que llama a `markSynchronized` y escribe S :

```
// SyncEngine.ts – registerRemoteFolder
this.ledgerManager.markSynchronized(vaultPath, { type: "folder", remoteId });
```

`markSynchronized` es incondicional: sobrescribe `C` con `S`. Y aunque el evento `vault.on("create")` llegue **después**, no puede revertirlo, porque `trackFolder` sale antes de tocar el estado:

```
// LedgerManager.ts - trackFolder
trackFolder(path: string): void {
  if (this.manifest.entries[path]) {
    return; // ← no degrada S → C
  }
  this.manifest.entries[path] = { type: "folder", status: "C" };
}
```

Resultado: **en cualquier orden de ejecución el estado final es `S`**.

1.3 ¿Realmente falta confirmación HTTP?

Parcialmente, y esa es la parte sutil. `resolveOrCreateFolderPath` **sí** hace `GET` + `POST` en camino frío, así que la carpeta existe en Drive. Pero el `S` es **falso a nivel de protocolo** por tres motivos:

1. **Caché sin red (H-05).** `folderPathCache` indexa por `rootFolderId: ruta`. Si la ruta se resolvió antes en la misma sesión, `resolveSubfolderExclusive` devuelve el id cacheado **sin una sola petición**. Si esa carpeta fue borrada en Drive entretanto, se otorga `S` con un `remoteId` muerto.
2. **El manifiesto remoto nunca se sube.** En el modelo v1.1.x, «sincronizado» significa *presente y consistente en `.obsave/ledger.json` remoto*. `syncTemplateFoldersToCloud` solo persiste el ledger **local**; la subida del manifiesto vive exclusivamente en `runLedgerSync`. Con auto-sync OFF ese ciclo no se ejecuta jamás.
3. **En GitHub el `S` puede no tener red detrás (H-03).** Ver §4.

1.4 Por qué el motor ignora después esas carpetas

El push solo recorre entradas pendientes:

```
// LedgerManager.pendingPaths - filtra C | U | D
pendingPaths(): string[] {
  return Object.entries(this.manifest.entries)
    .filter(([, e]) => e.status === "C" || e.status === "U" || e.status === "D")
    .map(([p]) => p);
}
```

Una carpeta en `S` **queda fuera de `pushLocalPendingChanges` para siempre**. Cuando el usuario activa la sincronización más tarde:

1. `hasPendingChanges()` puede ser `false` → si además el hash del manifiesto coincidiera, el ciclo **aborta en el Paso 1**.
2. Si el ciclo corre, `pushLocalPendingChanges` **salta** las carpetas (no son pendientes) y solo se sube el manifiesto completo al final.
3. Si el `remoteId` cacheado era obsoleto o la carpeta se creó bajo otra cuenta, nada la recrea: no hay verificación de existencia para entradas en `S`.

Agravante (H-04). `trackFolder` tampoco recupera una carpeta marcada `D`. Secuencia reproducible con auto-sync OFF:

Paso	Acción	Estado en ledger
1	Sync previa correcta	S
2	Usuario borra 05_Archivadas localmente	D (más hijos en D)
3	Auto-sync OFF → no se propaga	D
4	Usuario recrea 05_Archivadas	D (retorno temprano)
5	Usuario activa auto-sync	deleteRemotePath → borra la carpeta en Drive/GitHub

Es decir, recrear una carpeta puede provocar su **borrado remoto recursivo**.

2. Inconsistencia entre flujos con y sin auto-sync

2.1 Comparativa de rutas

Etapas	Auto-sync ACTIVA	Auto-sync DESACTIVADA
Creación local	trackFolder → C	trackFolder → C
Template sync (botón)	Se ejecuta → S + ledger.save()	Se ejecuta igual → S + ledger.save()
scheduleDebounceSync (3 s)	canAutoSync() true → executeUnifiedSync	canAutoSync() false → return silencioso
Manifiesto remoto	Se sube al cierre de runLedgerSync	Nunca se sube
lastSyncAt	Se actualiza	Permanece obsoleto
Convergencia	Local y remoto reconcilian	Divergencia permanente hasta un sync manual

La asignación C vs S **es idéntica en ambos flujos**; lo que cambia es la **reconciliación posterior**. Con auto-sync activa el ciclo enmascara el defecto de §1 porque sube el manifiesto 3 s después. Con auto-sync desactivada el S prematuro queda expuesto y es el estado final observable.

2.2 Tercer desenlace: Drive sin carpeta seleccionada

syncTemplateFoldersToCloud exige isGoogleDriveFolderReady(). Si el proveedor es Drive pero no hay carpeta elegida, la función **no hace nada y no informa** (cae hasta la rama github, que tampoco aplica, y retorna). Las carpetas quedan en C. La misma acción de usuario produce por tanto **tres desenlaces distintos** según estado en memoria: S con red, S desde caché, o C sin aviso.

2.3 Dependencias de memoria y entorno que alteran el resultado

Dependencia	Ámbito	Efecto sobre c/s
folderPathCache	Instancia GoogleDriveProvider	S sin HTTP; se vacía al recargar el plugin

targetFolderResolution	Instancia GoogleDriveProvider	Serializa la raíz; se pierde al recargar
GoogleDriveLazyProvider.delegate	Lazy	Primera llamada carga módulo y caché vacía → sí hay HTTP
remoteLedgerId	SyncEngine, en memoria	Tras recarga fuerza refetch del manifiesto
LedgerManager.loaded	En memoria	runLedgerSync recarga si es false
settings.autoSyncEnabled	data.json	No afecta a la asignación de estado, solo a la reconciliación

Hallazgo adicional (H-12): `requestStructureSync()` es código muerto. Ninguna ruta lo invoca; `structureSyncNeeded` solo se asigna a `false`. En consecuencia la rama de plantillas dentro de `runLedgerSync` **nunca se dispara** y el botón manual es la **única** vía que materializa las carpetas plantilla en la nube.

3. Integridad del manifiesto `.obsave/ledger.json`

3.1 Dos almacenes sin transacción común

Almacén	Ruta	Escritor	Contenido
Ajustes	<code>.obsidian/plugins/obsave/data.json</code>	<code>Plugin.saveData</code>	Credenciales, <code>autoSyncEnabled</code> , <code>syncedLedger</code> (legacy)
Manifiesto	<code>.obsidian/plugins/obsave/ledger.json</code>	<code>adapter.write</code>	Estados C/U/D/S, <code>remoteId</code> , hashes

No hay bloqueo, escritura atómica (temp + rename) ni versión de secuencia entre ambos.

3.2 Carreras identificadas

H-10 — `save()` concurrente sin cerrojo. Tres emisores escriben el mismo archivo sin coordinación: `persistLedger` (en cada evento del vault), `syncTemplateFoldersToCloud` y `runLedgerSync`. `JSON.stringify` es síncrono pero `adapter.write` es asíncrono, así que **una instantánea antigua puede aterrizar después de una nueva**. En `main.ts` las llamadas son *fire-and-forget*:

```
const persistLedger = (): void => { void this.ledgerManager.save(); };
// ...
this.app.vault.on("create", (file) => { void
this.onVaultCreate(file).then(persistLedger); })
```

H-21 — Degradación silenciosa ante JSON corrupto. Si una escritura concurrente trunca el archivo, `load()` captura la excepción y **arranca con un manifiesto vacío**, sin backup ni notificación:

```
} catch {
  this.manifest = createEmptyManifest(this.getDeviceName());
```

```
}
```

El daño se amplifica al combinarse con **H-01**: manifiesto vacío → sin pendientes → el ciclo copia el remoto → todo queda en **S** local **sin comprobar el disco**. La bóveda puede quedar «verde» y desincronizada de forma indefinida.

H-11 — S persistido antes de confirmar el manifiesto remoto. En `runLedgerSync` el orden es: marcar **S** → `ledgerManager.save()` → `uploadRemoteLedger()`. Si la subida falla, en disco ya consta **S**. El contenido de los archivos sí llegó a la nube, de modo que el impacto se limita al desfase del manifiesto, corregible en el ciclo siguiente por diferencia de hash.

H-07 — Desconexión asimétrica. `disconnectProvider` limpia `settings.syncedLedger` y anula el proveedor, pero **deja intacto `ledger.json`**. Al reconectar contra otra cuenta o repositorio persisten entradas **S** con `remoteId` de la cuenta anterior; el motor las considera sincronizadas y **nunca las sube**, o bien intenta `PATCH` sobre ids inexistentes y aborta el ciclo con error.

Sobre sobrescritura de pendientes por estados limpios: la protección principal funciona.

`applyRemoteManifestChanges` salta toda ruta cuyo estado local sea **C**, **U** o **D**, y `finalizeFilePush` conserva **U** si el hash cambió durante la transferencia. La brecha real no es que **S** pise a **C/U/D**, sino que **S se conceda sin respaldo verificable** (H-01, H-02, H-03, H-05).

4. Puntos ciegos y regresiones potenciales

4.1 H-01 (Crítica) — El manifiesto remoto en **S** bloquea toda descarga

```
// SyncEngine.applyRemoteManifestChanges
if (remoteEntry.status === "C" || remoteEntry.status === "U") {
  /* descarga real */
}
if (remoteEntry.status === "S") {
  this.ledgerManager.markSynchronized(path, remoteEntry); // ← sin tocar el disco
}
```

El dispositivo que sube **deja todas sus entradas en S** antes de publicar el manifiesto. Por tanto, un segundo dispositivo recibe un manifiesto donde **todo está en S**, y la rama de descarga (**C / U**) **jamás se activa**. Peor aún, `markSynchronized(path, remoteEntry)` copia `hash`, `mtime` y `size` que describen el disco de **otro equipo**.

Consecuencia: ObSave se comporta como respaldo unidireccional. Un equipo nuevo, o una bóveda restaurada, muestra badges verdes con archivos inexistentes y nunca los descarga. Esto también explica por qué la restauración solo funciona tras usar «Reparar / Reconstruir Bóveda Remota».

4.2 H-03 (Crítica) — **S** sin ninguna petición HTTP en GitHub

```
// SyncEngine.pushRemoteFolder — rama github
return entry.remoteId ?? path; // ← "remoteId" = ruta literal
```

Para una carpeta **no vacía** en GitHub no se ejecuta ninguna llamada, y el llamador hace `markSynchronized(path, { remoteId })`. La carpeta queda en **S verde** con un `remoteId` que es su

propia ruta. Al no existir concepto de carpeta en la API de contenidos de GitHub, el valor nunca podrá validarse ni servir para borrado o renombrado.

4.3 H-06 (Alta) — Falso verde en carpetas desconocidas

```
// FileStatusDecorator.resolveFolderStatus – cola de la función
if (hasPendingChildren) { return "modified"; }
return "synced"; // ← sin entrada en ledger → verde
```

Una carpeta ausente del manifiesto y sin hijos pendientes **se pinta verde**. Además `buildFolderPendingIndex` solo recorre entradas del ledger: si los hijos tampoco están registrados, no hay señal de pendiente. La UI afirma «sincronizada» sobre una entidad que el motor desconoce.

4.4 Inventario completo de S sin confirmación del proveedor

ID	Ubicación	Naturaleza del S falso
H-01	<code>applyRemoteManifestChanges</code> (rama S)	Copia del manifiesto ajeno sin verificar disco local
H-02	<code>registerRemoteFolder</code>	S local sin publicar manifiesto remoto
H-03	<code>pushRemoteFolder</code> (GitHub, carpeta con contenido)	Sin HTTP; <code>remoteId</code> = ruta
H-05	<code>resolveOrCreateFolderPath</code> con caché	Sin HTTP; id potencialmente muerto
H-09	<code>registerRemoteGitkeep</code>	<code>remoteId</code> de la carpeta = SHA del blob <code>.gitkeep</code>
H-13	<code>syncTemplateFoldersToGoogleDrive</code>	Crea las 6 plantillas en Drive aunque no existan localmente; <code>registerRemoteFolder</code> las descarta → carpetas remotas huérfanas fuera del manifiesto
—	<code>applyRemoteManifestChanges</code> (carpeta remota C/U)	<code>markSynchronized(..., { remoteId: remoteEntry.remoteId })</code> acepta undefined

4.5 Otros hallazgos del barrido

ID	Sev.	Hallazgo
H-08	Media-Alta	<code>deleteRemotePath</code> sin <code>remoteId</code> retorna en silencio, pero el llamador ejecuta <code>removeEntry + removeDescendantEntries</code> : el objeto permanece en la nube y desaparece del manifiesto → huérfano invisible
H-14	Media	Asimetría de proveedores: en Drive las 6 plantillas se reportan; en GitHub solo las vacías reciben <code>.gitkeep</code> y callback
H-16	Media	<code>clearGitHubStorage</code> borra todos los blobs del repositorio salvo <code>.obsidian/</code> y <code>.git/</code> — destructivo si el repo alberga otro contenido. En Drive, <code>clearRemoteStorage</code> usa DELETE permanente , no papelera

H-17	Media	Sin resolución de conflictos: si ambos lados cambian la misma ruta, el pendiente local gana y la versión remota se pierde sin copia de conflicto ni aviso
H-19	Media	<code>pullRemoteFile</code> (Drive) sin <code>remoteId</code> devuelve <code>false</code> ; el llamador ignora el fallo y la entrada queda inconsistente sin reintento
H-18	Baja-Media	<code>confirmPathDeleted</code> (GitHub) considera «existe» cualquier respuesta distinta de 404, incluidos 401/403/500 y errores de red → decisiones erróneas ante fallos transitorios
H-15	Baja-Media	<code>hashContent</code> es djb2 de 32 bits: espacio de colisión reducido para un control de integridad de contenido
H-20	Baja	<code>onunload</code> no fuerza un <i>flush</i> final del ledger; una escritura en vuelo puede perderse al cerrar

5. Plan de acción propuesto

P0 — Corrección inmediata (integridad de datos)

- H-01.** En `applyRemoteManifestChanges`, tratar la rama `S` como *candidata a descarga*: comprobar existencia y hash en disco; si el archivo falta o difiere, descargar. Para carpetas, garantizar `ensureLocalFolder`. Es el requisito para que el modo multi-dispositivo funcione.
- H-03 / H-09.** Modelar las carpetas de GitHub sin `remoteId` sintético: usar un marcador explícito (`remoteId: undefined` + bandera `remoteVerified`) y no otorgar `S` sin confirmación de la API.
- H-04.** En `trackFolder`, si la entrada existente está en `D`, promover a `C` en lugar de retornar.
- H-02.** Publicar el manifiesto tras `syncTemplateFoldersToCloud`, o bien registrar las carpetas como `C` con el `remoteId` conocido y dejar que el ciclo las consolide en `S`.

P1 — Robustez del manifiesto

- H-10 / H-21.** Serializar `save()` mediante cola de promesas, escribir a archivo temporal con *rename* atómico y conservar `ledger.json.bak`. Ante JSON inválido, restaurar el backup y notificar en lugar de degradar a vacío.
- H-07.** Reiniciar `ledger.json` en `disconnectProvider` y al cambiar de cuenta o repositorio.
- H-05.** Invalidar `folderPathCache` al inicio de cada ciclo, o revalidar el id con un `GET` ligero antes de conceder `S`.
- H-08.** No eliminar entradas del ledger si el borrado remoto no pudo ejecutarse; conservar `D` y reintentar.

P2 — Consistencia y experiencia

- H-06.** El decorador debe distinguir «sin datos» de «sincronizado»: carpeta desconocida →  rojo.
- H-12.** Eliminar `requestStructureSync()` o conectarlo a un disparador real.
- H-17.** Introducir detección de conflicto con copia `nombre (conflicto DISPOSITIVO).md`.
- H-16.** Acotar `clearRemoteStorage` a rutas presentes en el manifiesto y usar papelera en Drive (`trashed: true`).
- H-15.** Migrar el hash a SHA-256 vía `crypto.subtle`, con versión en el manifiesto para migración progresiva.

6. Metodología

- Revisión estática del árbol `src/` en el commit `afaa5a1` : `engine/SyncEngine.ts` , `ledger/{LedgerManager,RemoteManifestStore,manifestHash,types}.ts` , `main.ts` , `ui/{FileStatusDecorator,ObSaveSettingTab}.ts` , `productivity/{vaultStructure,vaultFolderSync}.ts` , `providers/{GoogleDriveProvider,GoogleDriveLazyProvider,GitHubProvider}.ts` , `oauth/GitHubProvider.ts` , `settings.ts` , `settingsMerge.ts` .
- Trazado de los flujos de eventos de Obsidian (`create` , `modify` , `delete` , `rename`) hasta la decisión de red.
- Verificación cruzada de invocadores mediante búsqueda de símbolos (`requestStructureSync` , `structureSyncNeeded` , `pendingPaths` , `markSynchronized`).
- Sin ejecución de pruebas E2E contra APIs en vivo; los hallazgos derivan de análisis de flujo de control y de estado.

Fin del informe — Auditoría holística ObSave v1.1.3